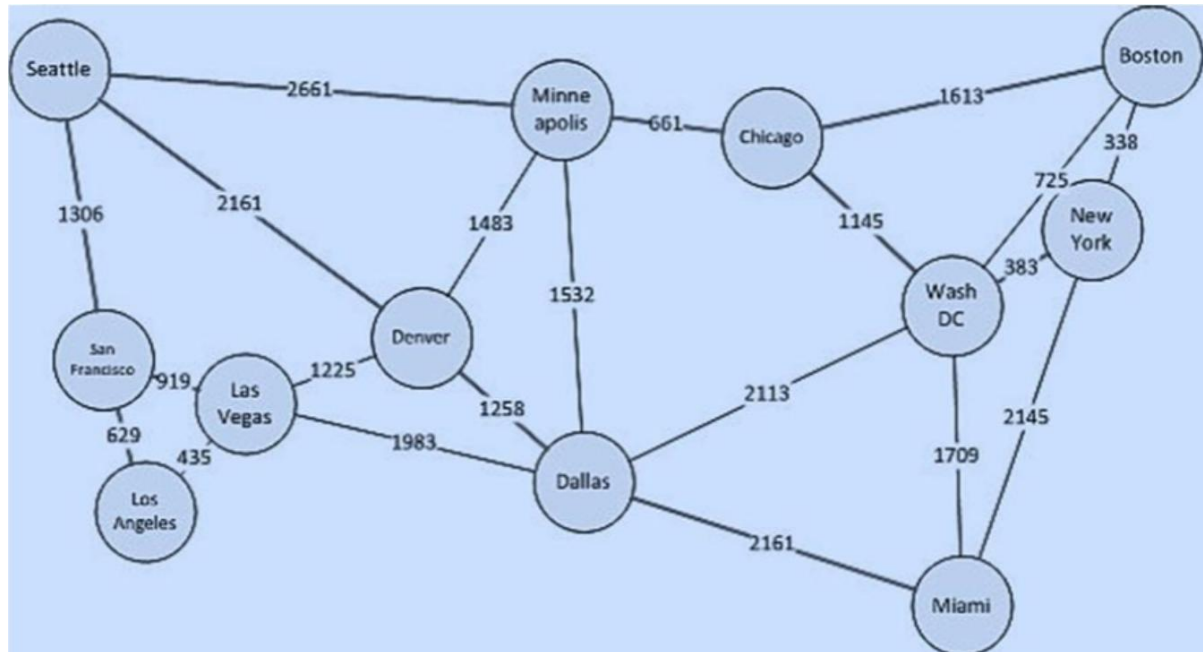


Dijkstra's Algorithm Implementation for Air Traffic Control

1. **Graph Representation:** The airports and flights are represented as a graph, where airports are nodes and flights are edges with weights (distances).



2. **Dijkstra's Algorithm:** This algorithm finds the shortest path from a starting node to all other nodes in a weighted graph. It uses a priority queue to explore the nearest unvisited node.
3. **Data Structures:**
 - `Map<Airport, List<Flight>>` graph: Represents the graph where each airport maps to a list of flights.
 - `Map<Airport, Double>` distances: Stores the shortest known distance from the start airport to each airport.
 - `Map<Airport, Airport>` previousNodes: Keeps track of the previous airport on the shortest path to reconstruct the route later.
 - `PriorityQueue<Airport>` priorityQueue: A priority queue to efficiently fetch the next airport with the shortest distance.

Code Structure

The DijkstraAlgorithm class is annotated with `@Service`, indicating that it is a Spring service component. The main method, `findShortestPath`, takes in a graph of airports and flights, along with a starting and ending airport, and returns a `Result` object containing the shortest path and its total distance.

Steps in the Code:

1. **Initialization:** Set all distances to infinity and the starting airport's distance to zero.
2. **Priority Queue Processing:** Continuously extract the airport with the smallest distance until the destination is reached or all reachable airports are processed.
3. **Path Reconstruction:** Once the destination is reached, backtrack using the `previousNodes` map to construct the path.
4. **Result Preparation:** Return the path and total distance in a `Result` object.

Program Implementation :

Dijkstra's Algorithm Implementation for Air Traffic Control

Overview

This Java implementation of Dijkstra's algorithm provides a robust solution for finding the shortest flight paths between airports. The code utilizes efficient data structures and a clear algorithmic approach to ensure optimal flight route planning.

Code Structure

The DijkstraAlgorithm class is annotated with `@Service`, indicating that it is a Spring service component. The main method, `findShortestPath`, takes in a graph of airports and flights, along with a starting and ending airport, and returns a `Result` object containing the shortest path and its total distance.

Steps in the Code

1. Initialization

- Set all distances to infinity and the starting airport's distance to zero.

```
for (Airport airport : graph.keySet()) {  
  
    distances.put(airport, Double.MAX_VALUE);  
  
    previousNodes.put(airport, null);  
  
}  
  
distances.put(start, 0.0);  
  
priorityQueue.add(start);
```

2. Priority Queue Processing

- Continuously extract the airport with the smallest distance until the destination is reached or all reachable airports are processed.

```
while (!priorityQueue.isEmpty()) {  
  
    Airport current = priorityQueue.poll();  
  
    // Early exit if we reach the destination  
  
    if (current.equals(end)) {  
  
        // Path reconstruction logic  
  
        reconstructPath(current, previousNodes, path);  
  
        break;  
  
    }  
  
    // Update neighbors (flights)  
  
    List<Flight> flights = graph.get(current);  
  
    for (Flight flight : flights) {  
  
        Airport neighbor = flight.getDestination();
```

```

double distance = distances.get(current) + flight.getDistance();

// Update distance and previous node if a shorter path is found

if (distance < distances.get(neighbor)) {

    distances.put(neighbor, distance);

    previousNodes.put(neighbor, current);

    priorityQueue.add(neighbor);

}

}

}

```

3. Path Reconstruction

- Once the destination is reached, backtrack using the previousNodes map to construct the path.

```

while (previousNodes.get(current) != null) {

    path.add(current);

    current = previousNodes.get(current);

}

path.add(start);

Collections.reverse(path);

```

4. Result Preparation

- Return the path and total distance in a Result object.

// Create a graph of airports and flights

```

Map<Airport, List<Flight>> graph = new HashMap<>();

```

// Find the shortest path from airport A to airport B

DijkstraAlgorithm algorithm = new DijkstraAlgorithm();

Result result = algorithm.findShortestPath(graph, airportA, airportB);

// Print the shortest path and its total distance

System.out.println("Shortest Path: " + result.getPath());

System.out.println("Total Distance: " + result.getDistance());